PYTHON

Module 20: While Loops & State Control

CODEHIVE

Iteration with While Loops

The `while loop is a fundamental control flow statement in Python that allows you to execute a block of code repeatedly as long as a specific condition remains True .`

`Logic at CodeHive:`

• **Condition-First**: Unlike loops that iterate over a collection, the `while loop focuses on a boolean state.`

• **Flexibility**: Ideal for scenarios where the number of iterations is not known in advance (e.g., waiting for user input or a specific system flag).

Structure & Safety

A standard while loop consists of an initializer, a condition, and an updater.

• **Infinite Loops**: If the condition never becomes False, the loop will run forever. This is a common bug. Always ensure the loop has an 'exit strategy' (like incrementing a counter).

Example: while i < 10: requires i to change inside the block.

Flow Control: break & continue

To manage complex iterations, Python provides keywords that override standard loop behavior:

- **break**: Immediately terminates the entire loop, jumping to the code following the loop block.

- **continue**: Skips the rest of the current iteration and jumps back to the top of the loop to re-evaluate the condition.

The Loop 'else' Statement

A unique feature of Python is the `else` block for loops.

- **Function**: The `else` block runs once when the loop condition becomes `False`.

- **The Break Exception**: If the loop is terminated by a `break` statement, the `else` block is skipped entirely. This is useful for searching operations where you want to run code only if a match was *not* found.

Best Practices for CodeHive Developers

- **Sentinels**: Use specific 'sentinel values' (like `q` for quit) to end user-driven loops.
- **Counters**: Standardize using `i += 1` syntax for clarity.
- **Readability**: If a `while` loop gets too complex, consider if it can be refactored into a `for` loop or separated into a function.